Qassim University جامعة القصيم

Semester 443 - 2023, 1444

Course CS315 Algorithms Analysis and Design

Dynamic programming

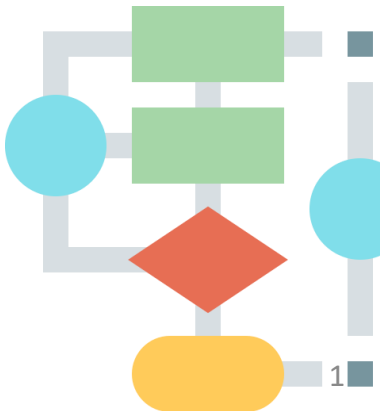
Lecture 4

Prof. Dr Mohammed Al-Hagery

Supervisordr@yahoo.com

Athar alwabel

aa.alwabel@qu.edu.sa

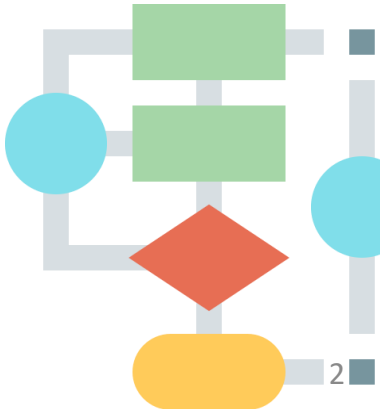


Dynamic programming

One of the most important algorithm tools!

Dynamic programming is an optimization method based on the principle of optimality defined by Bellman1 in the 1950s: “An optimal policy has the property that whatever the initial state and initial decision are, the remaining decisions must constitute an optimal policy with regard to the state resulting from the first decision.”

It can be summarized simply as follows: “every optimal policy consists only of optimal sub policies.”



Fibonacci numbers

1, 1, 2, 3, 5, 8, 13, 21, 34, …

What is the recurrence for the nth Fibonacci number?

$F(n) = F(n-1) + F(n-2)$

The solution for n is defined with respect to the solution to smaller problems

(n-1 and n-2)

Running time

```
FIBONACCI(n)
1  if n = 1 or n = 2
2          return 1
3  else
4          return FIBONACCI(n − 1) + FIBONACCI(n − 2)
```

$O(2^n)$

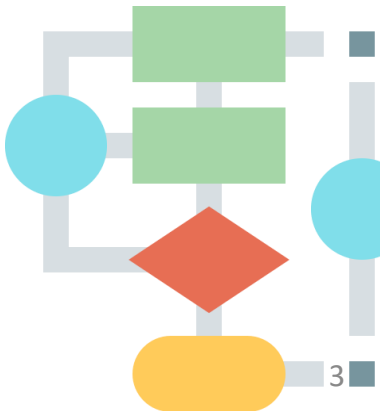
Can we do better?

A lot of repeated work !

Each call creates two recursive calls

Each call reduces the size of the problem by 1 or 2

Creates a full binary of depth n



Creating a dynamic programming solution

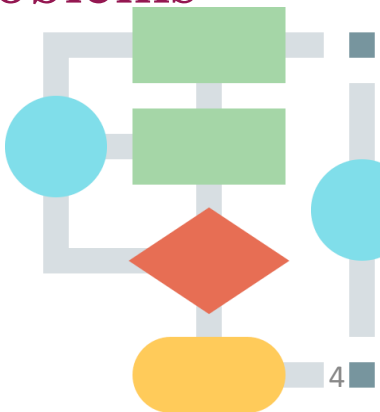
Step 1: Identify a solution to the problem with respect to smaller subproblems (pretend like you have a solver, but it only works on smaller problems):

F(n) = F(n-1) + F(n-2)

Step 2: bottom up - start with solutions to the smallest problems and build solutions to the larger problems

```
FIBONACCI-DP(n)
1  fib[1] ← 1
2  fib[2] ← 1
3  for i ← 3 to n
4        fib[i] ← fib[i − 1] + fib[i − 2]
5  return fib[n]
```

Use an array to store solutions to subproblems



Running time = O(n) for each part

Identifying a dynamic programming problem

The solution can be defined with respect to solutions to subproblems .

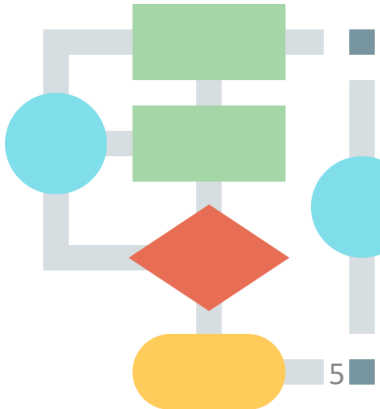
The subproblems created are overlapping, that is we see the same subproblems repeated

Dynamic programming Properties

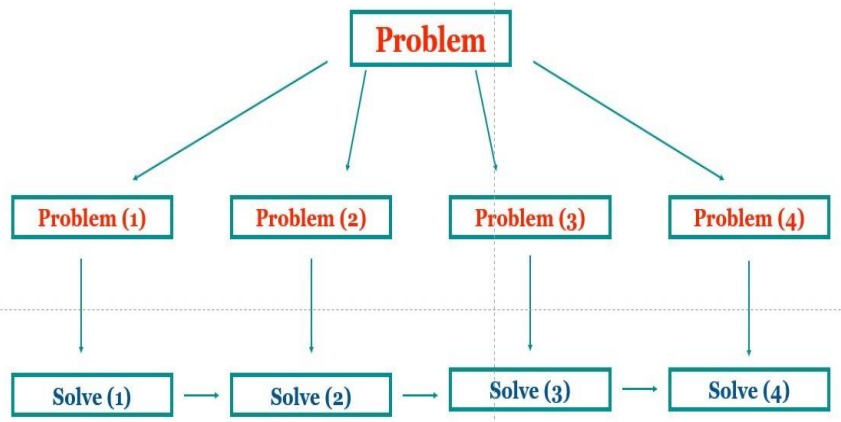
Dynamic programming works if a problem exhibit the following two Properties

1- Optimal sub-structure : An optimal solution to the problem contains within it, optimal solution to other sub-problems.

2- Overlapping sub-problems : When recursive algorithm revisits the same problem repeatedly, then the optimization problem has overlapping sub-problems.

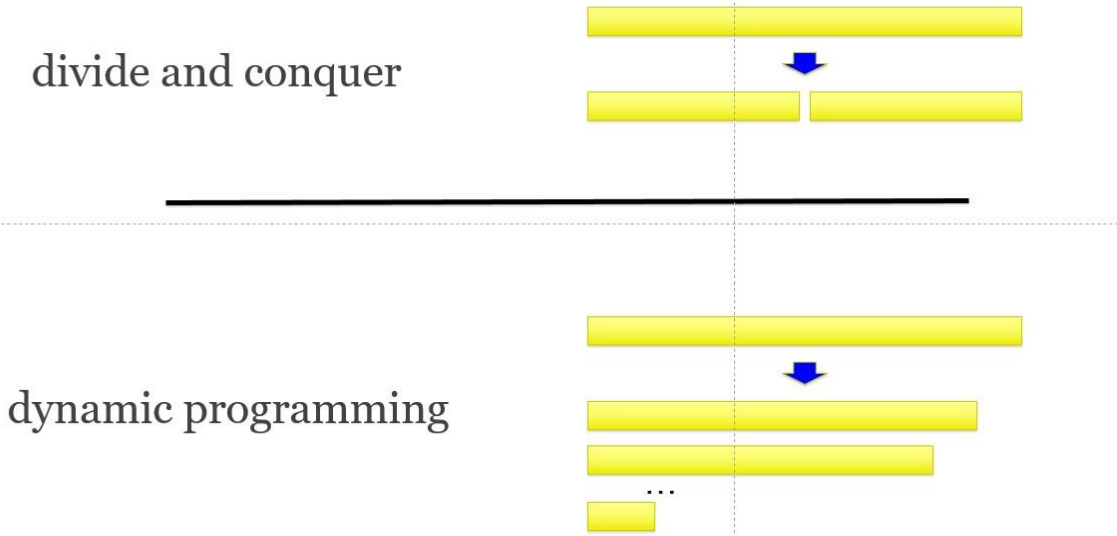


Dynamic programming



Dynamic programming we try to find all possible solution and take the best solution (Optimal solution)

Overlapping sub-problems



Dynamic programming and it's Application

When developing a dynamic-programming algorithm, we follow a sequence of four steps:

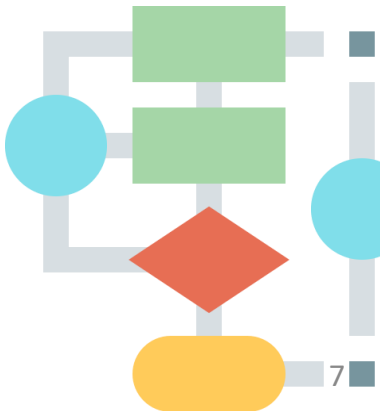
1. **Characterize the structure of an optimal solution.**
2. **Recursively define the value of an optimal solution.**
3. **Compute the value of an optimal solution, typically in a bottom-up fashion.**
4. **Construct an optimal solution from the solutions computed previously.**

The Application

1- Matrix-chain multiplication

2- Longest common subsequence

3- Optimal binary search trees.



Matrix-chain multiplication

Suppose we have a sequence or chain A1, A2, …, An of n matrices to be multiplied. That is, we want to compute the product A1A2…An. There are many possible ways (parenthesizations) to compute the product.

Example: consider the chain A1, A2, A3, A4 of 4 matrices.

Let us compute the product A1A2A3A4.

There are 5 possible ways:

(A1(A2(A3A4)))= ?

(A1((A2A3)A4))=?

((A1A2)(A3A4))=?

((A1(A2A3))A4)=?

(((A1A2)A3)A4)=?

To compute the number of scalar multiplications necessary, we must know:

Algorithm to multiply two matrices.

Matrix dimensions.

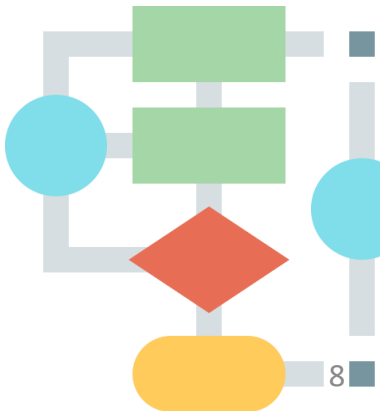
Number of scalar multiplications = pqr

Input: Matrices Ap×q and Bq×r

Output: Matrix Cp×r resulting from A·B

Algorithm Matrix-Multiply($Ap\times q$, $Bq\times r$)

```
{  for i := 1 to p do
     for j := 1 to r do
     {   C[i, j] := 0;
         for k := 1 to q do
    C[i, j] := C[i, j] + A[i, k] · B[k, j];
     }
   return C;
}
```



Matrix-chain multiplication

Example:

Consider three matrices $A_{10\times100}$, $B_{100\times5}$, and $C_{5\times50}$

There are 2 ways to parenthesize

$((A.B).C) = D_{10\times5} \cdot C_{5\times50}$

$AB \Rightarrow 10\cdot100\cdot5 = 5{,}000$ scalar multiplications,

$DC \Rightarrow 10\cdot5\cdot50 = 2{,}500$ scalar multiplications,

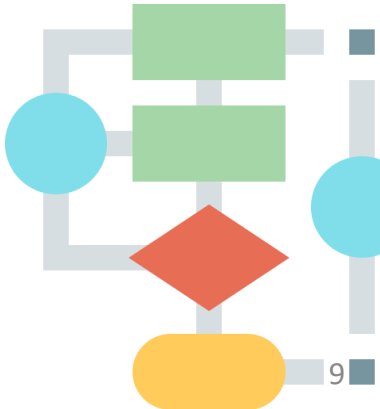
Total scalar multiplication = 7,500

$(A(BC)) = A_{10\times100} \cdot E_{100\times50}$

$BC \Rightarrow 100\cdot5\cdot50 = 25{,}000$ scalar multiplications,

$AE \Rightarrow 10\cdot100\cdot50 = 50{,}000$ scalar multiplications,

Total scalar multiplication = 75,000



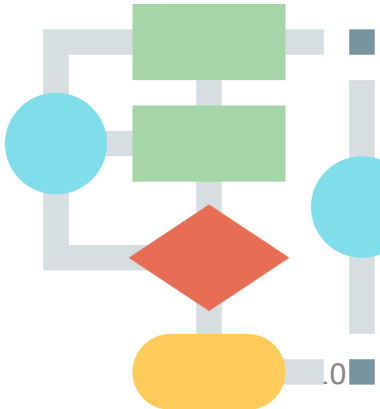
Matrix-chain multiplication algorithm

```
MATRIX-MULTIPLY(A, B)
1  if A.columns ≠ B.rows
2      error "incompatible dimensions"
3  else let C be a new A.rows × B.columns matrix
4      for i = 1 to A.rows
5          for j = 1 to B.columns
6              c_ij = 0
7              for k = 1 to A.columns
8                  c_ij = c_ij + a_ik · b_kj
9      return C
```

A 5X50 B 50X100

C 5X100

$$m[i,j] = \begin{cases} 0 & \text{if } i = j, \\ \min\limits_{i \le k < j} \{m[i,k] + m[k+1,j] + p_{i-1} p_k p_j\} & \text{if } i < j. \end{cases} \qquad (15.7)$$



Dynamic Prog.: Example

Matrices: $A_0 = 10x5$ $A_1 = 5x8$ $A_2 = 8x4$ $A_3 = 4x10$

	0	1	2	3
0	0	$A_0A_1 = 10*5*8$		
1		0		
2			0	
3				0

Dynamic Prog.: Example

Matrices: $A_0 = 10x5$ $A_1 = 5x8$ $A_2 = 8x4$ $A_3=4x10$

	0	1	2	3
0	0	A_0A_1=10*5*8 = 400		
1		0	?	
2			0	
3				0

Dynamic Prog.: Example

Matrices: $A_0 = 10x5$ $A_1 = 5x8$ $A_2 = 8x4$ $A_3=4x10$

	0	1	2	3
0	0	$A_0A_1=10*5*8$ $= 400$		
1		0	$A_1A_2=5*8*4$ $=160$	
2			0	?
3				0

Dynamic Prog.: Example

Matrices: $A_0 = 10x5$ $A_1 = 5x8$ $A_2 = 8x4$ $A_3=4x10$

	0	1	2	3
0	0	$A_0A_1=10*5*8$ $= 400$		
1		0	$A_1A_2= 5*8*4$ $=160$	
2			0	$A_2A_3=8*4*10=320$
3				0

$(A_0(A_1 A_2))$

Dynamic Prog.: Example

Matrices: $A_0 = 10x5$ $A_1 = 5x8$ $A_2 = 8x4$ $A_3=4x10$

	0	1	2	3
0	0	$A_0A_1=10*5*8 = 400$	$A_0A_1A_2=A_0(A_1A_2)=160+?$	
1		0	$A_1A_2=5*8*4=160$	
2			0	$A_2A_3 =8*4*10=320$
3				0

Dynamic Prog.: Example

Matrices: A_0 = 10x5 A_1 = 5x8 A_2 = 8x4 A_3=4x10

	0	1	2	3
0	0	A_0A_1=10*5*8 = 400	$A_0A_1A_2 = A_0(A_1A_2)$ = 160+10*5*4 = 160+200 = 360	
1		0	A_1A_2=5*8*4=160	
2			0	A_2A_3 =8*4*10=320
3				0

Dynamic Prog.: Example

Matrices: $A_0 = 10x5$ $A_1 = 5x8$ $A_2 = 8x4$ $A_3=4x10$

	0	1	2	3
0	0	$A_0A_1=10*5*8 = 400$	$A_0A_1A_2 = A_0(A_1A_2) = 160+10*5*4 = 160+200 = 360$ $(A_0A_1)A_2=400+\ldots$ no need…	
1		0	$A_1A_2=5*8*4=160$	
2			0	$A_2A_3 =8*4*10=320$
3				0

Dynamic Prog.: Example

Matrices: $A_0 = 10x5$ $A_1 = 5x8$ $A_2 = 8x4$ $A_3=4x10$

	0	1	2	3
0	0	$A_0A_1=10*5*8 = 400$	$A_0A_1A_2 = A_0(A_1A_2) =$ $160+10*5*4 = 160+200 =$ **360** $(A_0A_1)A_2=400+10*8*4=720$	
1		0	$A_1A_2= 5*8*4=160$	
2			0	$A_2A_3 =8*4*10=320$
3				0

Dynamic Prog.: Example

Matrices: $A_0 = 10x5$ $A_1 = 5x8$ $A_2 = 8x4$ $A_3=4x10$

	0	1	2	3
0	0	$A_0A_1=10*5*8 = 400$	$A_0(A_1A_2) = 360$	
1		0	$A_1A_2=5*8*4=160$	$A_1A_2A_3 = A_1(A_2A_3) = 320+?$
2			0	$A_2A_3 =8*4*10=320$
3				0

Dynamic Prog.: Example

Matrices: $A_0 = 10x5$ $A_1 = 5x8$ $A_2 = 8x4$ $A_3=4x10$

	0	1	2	3
0	0	$A_0A_1=10*5*8 = 400$	$A_0(A_1A_2) = 360$	
1		0	$A_1A_2= 5*8*4= 160$	$A_1A_2A_3 = A_1(A_2A_3) = 320+5*8*10=720$
2			0	$A_2A_3 =8*4*10=320$
3				0

Dynamic Prog.: Example

Matrices: $A_0 = 10x5$ $A_1 = 5x8$ $A_2 = 8x4$ $A_3=4x10$

	0	1	2	3
0	0	A_0A_1=10*5*8 = 400	$A_0(A_1A_2)$ = 360	
1		0	A_1A_2=5*8*4= 160	$A_1A_2A_3 = A_1(A_2A_3)$ = 320+5*8*10=720 $(A_1A_2)A_3$ = 160+5*4*10=**360**
2			0	A_2A_3 =8*4*10=320
3				0

Dynamic Prog.: Example

Matrices: $A_0 = 10x5$ $A_1 = 5x8$ $A_2 = 8x4$ $A_3=4x10$

	0	1	2	3
0	0	A_0A_1=10*5*8 = 400	$A_0(A_1A_2)$ = 360	$A_0A_1A_2A_3$ = ?
1		0	A_1A_2=5*8*4= 160	$(A_1A_2)A_3$ = **360**
2			0	A_2A_3 =8*4*10=320
3				0

Dynamic Prog.: Example

Matrices: A_0 = 10x5 A_1 = 5x8 A_2 = 8x4 A_3=4x10

	0	1	2	3
0	0	A_0A_1=10*5*8 = 400	$A_0(A_1A_2)$ = 360	$A_0A_1A_2A_3$ = $A_0(A_1A_2A_3)$=360+10*5*10=860 $(A_0A_1A_2)A_3$=360+10*4*10=**760**
1		0	A_1A_2=5*8*4=160	$(A_1A_2)A_3$ = **360**
2			0	A_2A_3 =8*4*10=320
3				0

Dynamic Prog.: Example

Matrices: $A_0 = 10x5$ $A_1 = 5x8$ $A_2 = 8x4$ $A_3=4x10$

	0	1	2	3
0	0	$A_0A_1=10*5*8 = 400$	$A_0(A_1A_2) = 360$	$A_0A_1A_2A_3 =$ $A_0(A_1A_2A_3)=360+10*5*10=860$ $(A_0A_1A_2)A_3=360+10*4*10=$**760** $(A_0A_1)(A_2A_3)=400+360+ ? =$ *no need*
1		0	$A_1A_2=5*8*4=160$	$(A_1A_2)A_3 =$ **360**
2			0	$A_2A_3 =8*4*10=320$
3				0

Dynamic Prog.: Example

Matrices: $A_0 = 10x5$ $A_1 = 5x8$ $A_2 = 8x4$ $A_3=4x10$

	0	1	2	3
0	0	$A_0A_1=10*5*8=400$	$A_0(A_1A_2) = 360$	$A_0A_1A_2A_3 =$ $A_0(A_1A_2A_3)=360+10*5*10=860$ $(A_0A_1A_2)A_3=360+10*4*10=\mathbf{760}$ $(A_0A_1)(A_2A_3)=400+360+10*8*10=1560$
1		0	$A_1A_2=5*8*4=160$	$(A_1A_2)A_3 = \mathbf{360}$
2			0	$A_2A_3 =8*4*10=320$
3				0

Dynamic Prog.: Example

Matrices: $A_0 = 10x5$ $A_1 = 5x8$ $A_2 = 8x4$ $A_3=4x10$

	0	1	2	3
0	0	$A_0A_1=10*5*8 = 400$	$A_0(A_1A_2) = 360$	$\mathbf{(A_0(A_1A_2))A_3}=360+10*4*10=\mathbf{760}$
1		0	$A_1A_2=5*8*4=160$	$(A_1A_2)A_3 = \mathbf{360}$
2			0	$A_2A_3 =8*4*10=320$
3				0

Final Order: $\mathbf{(A_0(A_1A_2))A_3}$

Dynamic Prog: Example

Matrices: $A_0 = 10x5$ $A_1 = 5x8$ $A_2 = 8x4$ $A_3=4x10$ $A_4=10*6$

	0	1	2	3	4
0	0	$A_0 A_1$			
1		0	$A_1 A_2$		
2			0	$A_2 A_3$	
3				0	$A_3 A_4$
4					0

Dynamic Prog.: Example

Matrices: A_0 = 10x5 A_1 = 5x8 A_2 = 8x4 A_3=4x10 A_4=10*6

	0	1	2	3	4
0	0	A_0A_1=400			
1		0	A_1A_2=160		
2			0	A_2A_3=320	
3				0	A_3A_4=240
4					0

Dynamic Prog.: Example

Matrices: A_0 = 10x5 A_1 = 5x8 A_2 = 8x4 A_3=4x10 A4=10*6

	0	1	2	3	4
0	0	A_0A_1=400	$A_0(A_1A_2)$=160+10*5*4 =360 $(A_0A_1)A_2$=400+.. *No need*		
1		0	A_1A_2=160		
2			0	A_2A_3=320	
3				0	A_3A_4=240
4					0

Dynamic Prog.: Example

Matrices: A_0 = 10x5 A_1 = 5x8 A_2 = 8x4 A_3=4x10 A4=10*6

	0	1	2	3	4
0	0	A_0A_1=400	$A_0(A_1A_2)$=360 0		
1		0	A_1A_2=160	$A_1A_2A_3 = A_1(A_2A_3)$ = 320+5*8*10=720 $(A_1A_2)A_3$ = 160+5*4*10=**360**	
2			0	A_2A_3=320	
3				0	A_3A_4=240
4					0

Dynamic Prog.: Example

Matrices: A_0 = 10x5 A_1 = 5x8 A_2 = 8x4 A_3=4x10 A_4=10x6

	0	1	2	3	4
0	0	A_0A_1=400	$A_0(A_1A_2)$=360		
1		0	A_1A_2=160	$(A_1A_2)A_3$ = **360**	
2			0	A_2A_3=320	$A_2A_3A_4 = A_2(A_3A_4)$ = 240+8*4*6=432 $(A_2A_3)A_4$ = 320+8*10*6=...
3				0	A_3A_4=240
4					0

Dynamic Prog.: Example

Matrices: $A_0 = 10x5$ $A_1 = 5x8$ $A_2 = 8x4$ $A_3=4x10$ $A_4=10*6$

	0	1	2	3	4
0	0	$A_0A_1=400$	$A_0(A_1A_2)$ $=360$	$A_0(A_1A_2A_3)=360+10*5*10$ $(A_0A_1A_2)A_3=360+10*4*10$ $=760$	
1		0	$A_1A_2=160$	$(A_1A_2)A_3$ = **360**	
2			0	$A_2A_3=320$	$A_2(A_3A_4)$ = 432
3				0	$A_3A_4=240$
4					0

Dynamic Prog.: Example

Matrices: $A_0 = 10x5$ $A_1 = 5x8$ $A_2 = 8x4$ $A_3=4x10$ $A_4=10*6$

	0	1	2	3	4
0	0	$A_0A_1=400$	$A_0(A_1A_2)=360$	$A_0(A_1A_2A_3)$ $(A_0A_1A_2)A_3=760$ $(A_0A_1)(A_2A_3)=400+320+10*8*10$	
1		0	$A_1A_2=160$	$(A_1A_2)A_3 = \mathbf{360}$	
2			0	$A_2A_3=320$	$A_2(A_3A_4) = 432$
3				0	$A_3A_4=240$
4					0

Dynamic Prog.: Example

Matrices: $A_0 = 10x5$ $A_1 = 5x8$ $A_2 = 8x4$ $A_3=4x10$ $A_4=10*6$

	0	1	2	3	4
0	0	$A_0A_1=400$	$A_0(A_1A_2)$ =360	$(A_0A_1A_2)A_3$= 760	
1		0	$A_1A_2=160$	$(A_1A_2)A_3$ = **360**	$A_1(A_2A_3A_4)=432+5*8*6=672$ $(A_1A_2A_3)A_4=360+5*10*6=660$ $(A_1A_2)(A_3A_4)=160+240+?$ 5 X 4 X 6
2			0	$A_2A_3=320$	$A_2(A_3A_4)$ = 432
3				0	$A_3A_4=240$
4					0

Dynamic Prog.: Example

Matrices: A_0 = 10x5 A_1 = 5x8 A_2 = 8x4 A_3=4x10 A_4=10*6

	0	1	2	3	4
0	0	A_0A_1=400	$A_0(A_1A_2)$=360	$(A_0A_1A_2)A_3$ =760	
1		0	A_1A_2=160	$(A_1A_2)A_3$ = **360**	$(A_1A_2A_3)A_4$=660 $(A_1A_2)(A_3A_4)$=160+240+5*4*6=520
2			0	A_2A_3=320	$A_2(A_3A_4)$ = 432
3				0	A_3A_4=240
4					0

Dynamic Prog.: Example

Matrices: A_0 = 10x5 A_1 = 5x8 A_2 = 8x4 A_3=4x10 A_4=10*6

	0	1	2	3	4
0	0	A_0A_1= 400	$A_0(A_1A_2)$= 360	$(A_0A_1A_2)A_3$ =760	$(A_0A_1A_2A_3)A_4$=760+10*10*6 **$A_0(A_1A_2A_3A_4)$=520+10*5*6 = 820** $(A_0A_1)(A_2A_3A_4)$=400+432+10*8*6 = *no need* $(A_0A_1A_2)(A_3A_4)$=360+240+10*4*6 = 840
1		0	A_1A_2=160	$(A_1A_2)A_3$ = **360**	$(A_1A_2)(A_3A_4)$=520
2			0	A_2A_3=320	$A_2(A_3A_4)$ = 432
3				0	A_3A_4=240
4					0

Dynamic Prog.: Example

Matrices: A_0 = 10x5 A_1 = 5x8 A_2 = 8x4 A_3=4x10 A_4=10*6

	0	1	2	3	4
0	0	A_0A_1=400	$A_0(A_1A_2)$ =360	$(A_0A_1A_2)A_3$= 760	**$A_0(A_1A_2A_3A_4)$= 820**
1		0	A_1A_2=160	$(A_1A_2)A_3$ = **360**	$(A_1A_2)(A_3A_4)$=520
2			0	A_2A_3=320	$A_2(A_3A_4)$ = 432
3				0	A_3A_4=240
4					0

Final Order: **$A_0((A_1A_2)(A_3A_4))$**

Additional examples
Matrix-chain multiplication

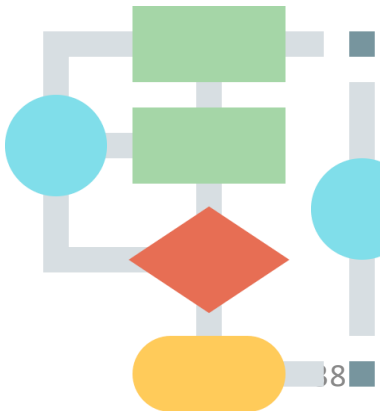
Example: 1

Consider the following six matrix problem.

Matrix	A_1	A_2	A_3	A_4	A_5	A_6
Dimensions	10x20	20x5	5x15	15x50	50x10	10x15

The problem therefore can be phrased as one of filling in the following table representing the values m.

i/j	1	2	3	4	5	6
1	0	1000				
2		0				
3			0			
4				0		
5					0	
6						0



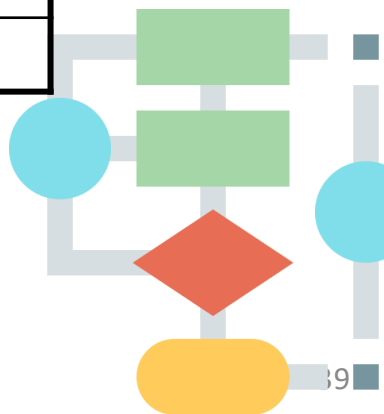
Matrix-chain multiplication

Example: 2

Chains of length 2 are easy, as there is no minimization required, so
m[i, i+1] = pi-1pipi+1

m[1, 2] = 10x20x5 = 1000
m[2, 3] = 20x5x15 = 1500
m[3, 4] = 5x15x50 = 3750
m[4, 5] = 15x50x10 = 7500
m[5, 6] = 50x10x15 = 7500

i/j	1	2	3	4	5	6
1	0	1000				
2		0	1500			
3			0	3750		
4				0	7500	
5					0	7500
6						0



Matrix-chain multiplication

Qassim University جامعة القصيم

Example: 3

Matrix	A_1	A_2	A_3	A_4	A_5	A_6
Dimensions	10x20	20x5	5x15	15x50	50x10	10x15

Chains of length 3 require some minimization – but only one each.

$A_1 (A_2 A_3)$ $(A_1 A_2) A_3$

$m[1,3]=\min\{(m[1,1]+m[2,3]+p_0p_1p_3),(m[1,2]+m[3,3]+p_0p_2p_3)\}$

$= \min\{(0+1500+10x20x15), (1000+0+10x5x15)\}$

$= \min\{4500, 1750\} = 1750$

$m[2,4]=\min\{(m[2,2]+m[3,4]+p_1p_2p_4),(m[2,3]+m[4,4]+p_1p_3p_4)\}$

$= \min\{(0+3750+20x5x50), (1500+0+20x15x50)\}$

$= \min\{8750, 16500\} = 8750$

$(A_3 A_4) A_5$

$m[3,5]=\min\{(m[3,3]+m[4,5]+p_2p_3p_5),(m[3,4]+m[5,5]+p_2p_4p_5)\}$

$= \min\{(0+7500+5x15x10), (3750+0+5x50x10)\}$

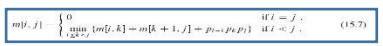
$= \min\{8250, 6250\} = 6250$

$m[4,6]=\min\{(m[4,4]+m[5,6]+p_3p_4p_6),(m[4,5]+m[6,6]+p_3p_5p_6)\}$

$= \min\{(0+7500+15x50x15), (7500+0+15x10x15)\}$

$= \min\{18750, 9750\} = 9750$

i/j	1	2	3	4	5	6
1	0	1000	1750			
2		0	1500	8750		
3			0	3750	6250	
4				0	7500	9750
5					0	7500
6						0



Matrix-chain multiplication

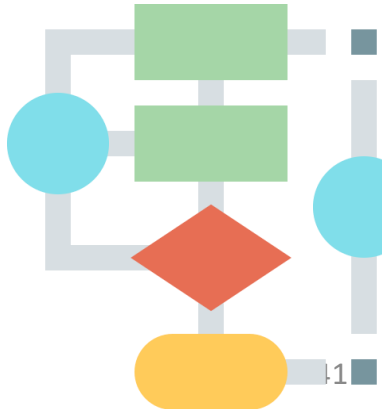
Chains of length 4 require some minimization – but only one each.

m[1,4]=min{(m[1,1]+m[2,4]+p0p1p4),(m[1,2]+m[3,4]+p0p2p4),
(m[1,3]+m[4,4]+p0p3p4)}
= min{(0+8750+10x20x50), (1000+3750+10x5x50),
(1750+0+10x15x50)}
= min { 18750, 7250, 9250 } = 7250

m[2,5]=min{(m[2,2]+m[3,5]+p1p2p5),(m[2,3]+m[4,5]+p1p3p5),
(m[2,4]+m[5,5]+p1p4p5)}
= min{(0+6250+20x5x10), (1500+7500+20x15x10),
(8750+0+20x50x10)}
= min { 7250, 12000, 18750 } = 7250

m[3,6]=min{(m[3,3]+m[4,6]+p2p3p6),(m[3,4]+m[5,6]+p2p4p6),
(m[3,5]+m[6,6]+p2p5p6)}
= min{(0+9750+5x15x15), (3750+7500+5x50x15),
(6250+0+5x10x15)}
= min { 10875, 15000, 7000 } = 7000

i/j	1	2	3	4	5	6
1	0	1000	1750	7250		
2		0	1500	8750	7250	
3			0	3750	6250	7000
4				0	7500	9750
5					0	7500
6						0



Matrix-chain multiplication

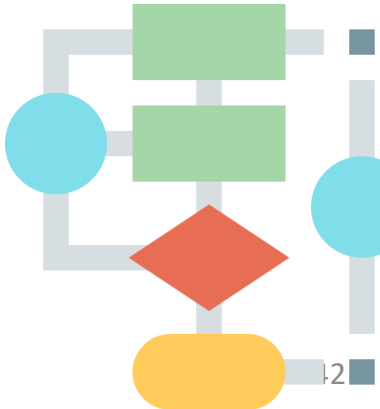
Chains of length 5 require some minimization – but only one each.

m[1,5]=min{(m[1,1]+m[2,5]+p0p1p5),(m[1,2]+m[3,5]+p0p2p5),
(m[1,3]+m[4,5]+p0p3p5),(m[1,4]+m[5,5]+p0p4p5)}
= min{(0+7250+10x20x10), (1000+6250+10x5x10),
(1750+7500+10x15x10), (7250+0+10x50x10)}
= min { 9250, 7750, 10750, 12250 } = 7750

m[2,6]=min{(m[2,2]+m[3,6]+p1p2p6),(m[2,3]+m[4,6]+p1p3p6),
(m[2,4]+m[5,6]+p1p4p6),(m[2,5]+m[6,6]+p1p5p6)}
= min{(0+7000+20x5x15), (1500+9750+20x15x15),
(8750+7500+20x50x15), (7250+0+20x10x15)}
= min { 8500, 15750, 31,250, 10250 } = 8500

m[1,6]=min{(m[1,1]+m[2,6]+p0p1p6),(m[1,2]+m[3,6]+p0p2p6),
(m[1,3]+m[4,6]+p0p3p6),(m[1,4]+m[5,6]+p0p4p6), (m[1,5]+m[6,6]+p0p5p6)}
= min{(11500, 8750, 13750, 22250, 9250 } = 8750

i/j	1	2	3	4	5	6
1	0	1000	1750	7250	7750	8750
2		0	1500	8750	7250	8500
3			0	3750	6250	7000
4				0	7500	9750
5					0	7500
6						0



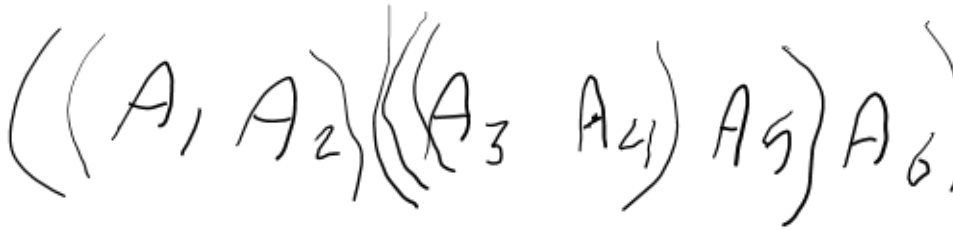
Example:

So far we have decided that the best way to parenthesize the expression results in 8750 multiplication.
But we have not addressed how we should actually DO the multiplication to achieve the value.
However, look at the last computation we did – the minimum value came from computing
A = (A1A2)(A3A4A5A6)
Therefore in an auxiliary array, we store value s[1,6]=2.
In general, as we proceed with the algorithm, if we find that the best way to compute Ai..j is as

Ai..j = Ai..kA(k+1)..j

then we set s[i, j] = k.

Then from the values of k we can reconstruct the optimal way to parenthesize the expression.
If we do this then we find that the s array looks like this:



i/j	1	2	3	4	5	6
1	1	1	2	2	2	2
2		2	2	2	2	2
3			3	3	4	5
4				4	4	5
5					5	5
6						6

We already know that we must compute A1..2 and A3..6.
By looking at s[3,6] = 5, we discover that we should compute A3..6 as A3..5 A6..6 and then by seeing that s[3,5] = 4,
we get the final parenthesization

A = ((A1A2)(((A3A4)A5)A6)).
And quick check reveals that this indeed takes the required 8750 multiplications.

Longest common subsequence (LCS)

For a sequence X = x1, x2, …, xn, a subsequence is a subset of the sequence defined by a set of increasing indices (i1, i2, …, ik) where $1 \le i1 < i2 < \ldots < ik \le n$

X = A B A C D A B A B

ABA? ACA? DCA? AADAA?

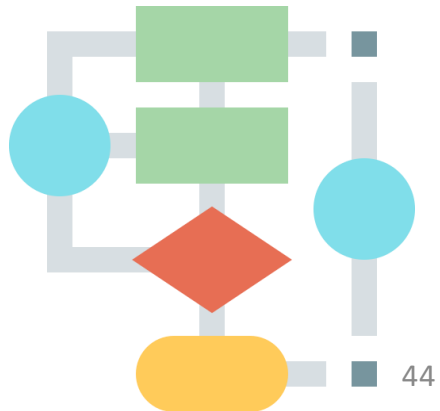
LCS problem

Given two sequences X and Y, a common subsequence is a subsequence that occurs in both X and Y Given two sequences X = x1, x2, …, xn and Y = y1, y2, …, yn,

X = A B C B D A B

Y = B D C A B A

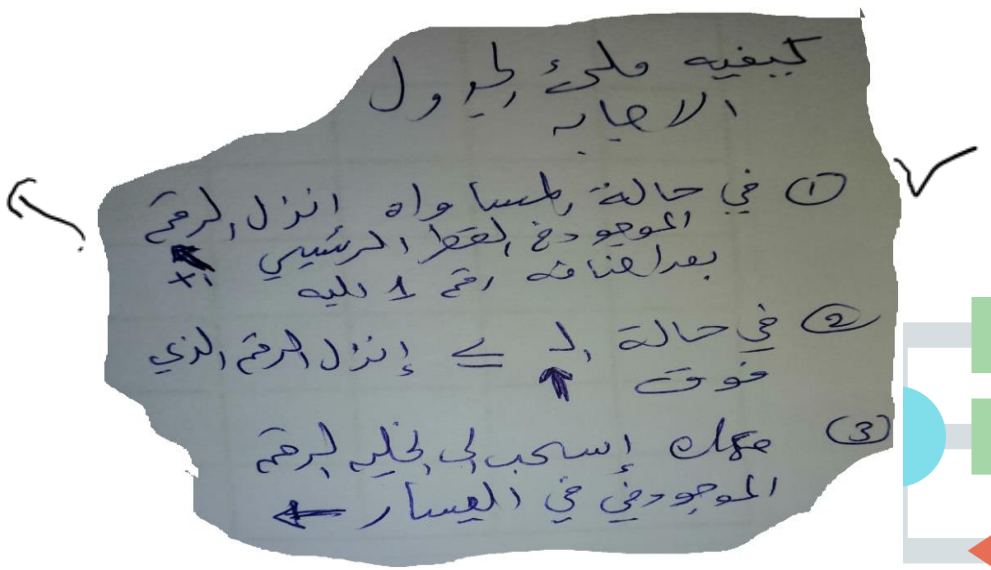
What is the longest common subsequence?



LCS Algorithm

```
LCS-LENGTH(X, Y)
 1  m = X.length
 2  n = Y.length
 3  let b[1..m, 1..n] and c[0..m, 0..n] be new tables
 4  for i = 1 to m
 5      c[i, 0] = 0
 6  for j = 0 to n
 7      c[0, j] = 0
 8  for i = 1 to m
 9      for j = 1 to n
10          if x_i == y_j
11              c[i, j] = c[i − 1, j − 1] + 1
12              b[i, j] = "↖"
13          elseif c[i − 1, j] ≥ c[i, j − 1]
14              c[i, j] = c[i − 1, j]
15              b[i, j] = "↑"
16          else c[i, j] = c[i, j − 1]
17              b[i, j] = "←"
18  return c and b
```

$$c[i,j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0\,, \\ c[i-1,j-1]+1 & \text{if } i,j > 0 \text{ and } x_i = y_j\,, \\ \max(c[i,j-1], c[i-1,j]) & \text{if } i,j > 0 \text{ and } x_i \neq y_j\,. \end{cases} \quad (15.9)$$



LCS Algorithm

Example:

X = A B C B D A B

Y = B D C A B A

i		j	0	1	2	3	4	5	6
		y_j	B	D	C	A	B	A	
0	x_i		0	0	0	0	0	0	0
1	A		0	0	0	0	1	1	1
2	B		0	1	1	1	1	2	2
3	C		0	1	1	2	2	2	2
4	B		0	1	1	2	2	3	3
5	D		0	1	2	2	2	3	3
6	A		0	1	2	2	3	3	4
7	B		0	1	2	2	3	4	4

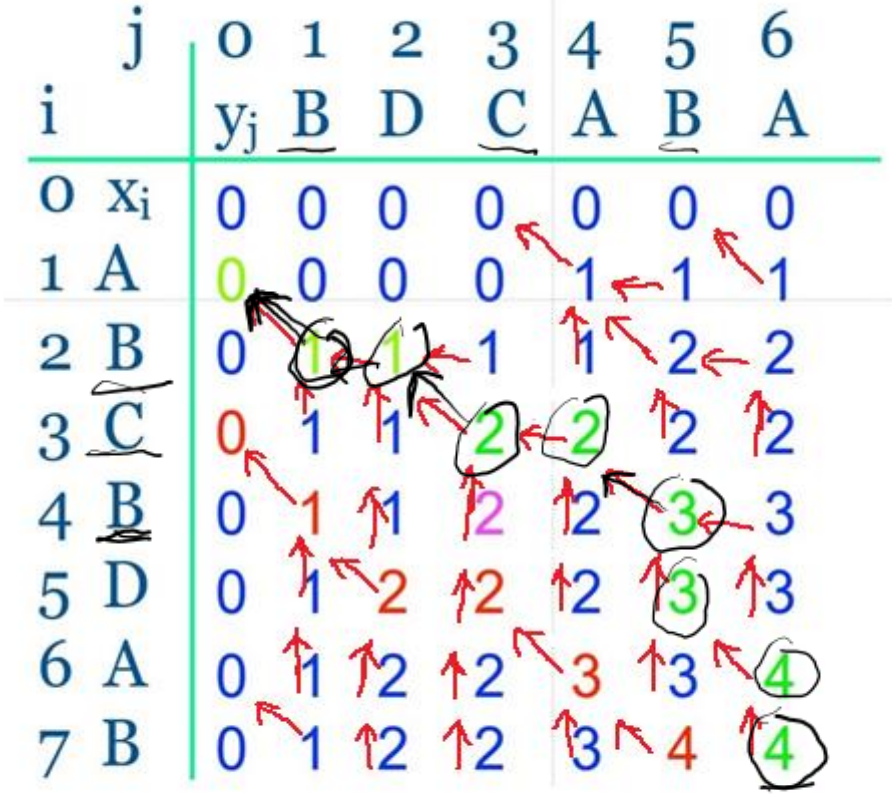
$$c[i,j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0, \\ c[i-1, j-1] + 1 & \text{if } i, j > 0 \text{ and } x_i = y_j, \\ \max(c[i, j-1], c[i-1, j]) & \text{if } i, j > 0 \text{ and } x_i \neq y_j. \end{cases} \quad (15.9)$$

LCS Algorithm

X = A B C B D A B

Example:

```
PRINT-LCS(b, X, i, j)
1  if i == 0 or j == 0
2        return
3  if b[i, j] == "↖"
4        PRINT-LCS(b, X, i − 1, j − 1)
5        print x_i
6  elseif b[i, j] == "↑"
7        PRINT-LCS(b, X, i − 1, j)
8  else PRINT-LCS(b, X, i, j − 1)
```



i	j	0	1	2	3	4	5	6
	y_j	B	D	C	A	B	A	
0	x_i	0	0	0	0	0	0	0
1	A	0	0	0	0	1	1	1
2	B	0	1	1	1	1	2	2
3	C	0	1	1	2	2	2	2
4	B	0	1	1	2	2	3	3
5	D	0	1	2	2	2	3	3
6	A	0	1	2	2	3	3	4
7	B	0	1	2	2	3	4	4

We can follow the arrows to generate the solution

BCBA

BDAB

BCAB

BCBA

4

https://www.youtube.com/watch?v=sSno9rV8Rhg

https://www.youtube.com/watch?v=jHGgXV27qtk

Additional solved example with the code

The End ..

This lecture was from

Introduction_to_Algorithms_Third_Edition_(2009)

chapter 15

